

作业感受

SGLang 在线推理服务挑战 · 从源码构建到 Mooncake trace 压测的完整实践记录

一、我是如何完成这项任务的

整体思路是先“把不确定性摊开”，再按依赖顺序逐个击破。动手之前我做的第一件事不是敲命令，而是把任务拆成**四条彼此独立的链路**：① 环境与依赖、② 源码构建、③ 服务与验证、④ 压测与交付。然后刻意让每条链路都尽可能少依赖外部网络——因为我很早就发现本机装了 Watt Toolkit 的 GitHub 加速，它做的是 TLS 中间人，会把 `git clone`、`curl` 这类走系统证书库的请求 直接打断。

于是我把 Mooncake trace、SGLang v0.5.14 源码包、standalone CPython 3.12、以及 Qwen3-0.6B 的 1.5 GB 权重**全部提前在 Windows 侧取好**，落到 `D:\first-task\work\`，再让 WSL 里的脚本“本地优先、网络兜底”。这一步之后，整个构建过程对外网的依赖降到了零，后面所有调试都不再被网络问题干扰。事后看，这是把后续难度砍掉一半的关键决定。

环境上还有一个容易忽略的点：Ubuntu 26.04 自带 Python 3.14，而 SGLang / Ray 的官方 CPU 配方 都基于 3.12。我没有去动系统 Python，而是解压了一份 standalone CPython 3.12 并用 `uv` 建了两个彼此隔离的虚拟环境——SGLang 一个、Ray 一个，两者只通过 HTTP 通信，正好对应任务里“可放在不同 Python 环境”的要求。

二、最困难的部分，以及我是怎么爬出来的

如果只能挑一件事，那就是下面这个坑。它几乎让我以为“这条路根本走不通”。

现象：一切看起来都成功了——apt 依赖装齐、torch 2.12.0+cpu 装好、SGLang 源码编译通过、`sgl-kernel` 的 C++ 也编译出了 `common_ops.cpython-312-x86_64-linux-gnu.so`。但一执行 `import sgl_kernel`，进程直接 `Illegal instruction (core dumped)` 退出，Python 层看不到任何异常栈。

定位过程（四步）

- 先确定“是谁在崩”**。分别单独 `import torch` 与 `sgl_kernel`，前者正常、后者崩溃——缩小到自编译的这一个扩展模块。
- 再确定“崩在什么指令上”**。用 `objdump -d` 反汇编那个 `.so`，统计出 **98672 处 `zmm` 寄存器引用**，即大量 AVX-512 指令。查 `/proc/cpuinfo`：这颗 Intel Core Ultra 5 338H（代号 Panther Lake，Intel Core Ultra Series 3）**只有 AVX2 与 AVX-VNNI，没有 AVX-512，也没有 AMX**——指令集不匹配，SIGILL 完全解释了。
- 再回到源码问“为什么会编成 AVX-512”**。在 `sgl-kernel/csrc/cpu/CMakeLists.txt` 中找到答案：对所有 x86_64 目标 **无条件加入** `-march=x86-64-v4 -mavx512bf16 -mavx512vnni -mamx-tile -mamx-bf16 -mamx-int8`。也就是说，这个“CPU 后端”实际上是给 Intel Xeon（Sapphire Rapids 及以上）写的，官方 `docker/xeon.Dockerfile` 也印证了这一点，而且**没有任何运行时指令集分派**。
- 最后判断“有没有救”**。关键在 `csrc/cpu/vec.h` 的前 5 行：`#if defined(__AVX512F__) && defined(__AVX512BF16__) && defined(__AMX_BF16__)` 才定义 `CPU_CAPABILITY_AVX512`。这说明 AVX-512 代码是被宏守卫包起来的，而 `vec.h` 里确实存在通用回退实现（例如 `vec_reduce_sum/max` 的 `#else` 分支）。**只要不以 v4 编译，这些宏自然失效，代码就会落到通用路径上**——于是有了救。

解决方式

我写了一个补丁脚本 `05_patch_cpu_isa.sh`，把那段 CMake 改成“可由 `SGLANG_CPU_ISA=baseline` 切换到 `-march=x86-64-v3`（AVX2 基线）”，**默认行为保持不变**（仍走上游 v4），原文件备份为 `CMakeLists.txt.orig` 可随时回退。同时让环境脚本用 `/proc/cpuinfo` 自动探测 AVX-512，没有就自动走基线、有就保持上游。重新编译后 `import sgl_kernel` 通过，服务正常拉起。

代价也必须说清楚：AVX2 基线下，少量依赖 AVX-512/AMX 的量化 GEMM（fp8 / int4 / int8）与 VNNI 优化路径不可用，会走通用实现，绝对吞吐低于 Xeon。但 bf16/fp32 的正常推理路径完整可用，对本次任务（Qwen3-0.6B bf16）没有影响。

顺带踩到的几个“小”坑

坑	根因与修法
脚本莫名以“unbound variable”退出	Shell 里 <code>MODEL_ID=“...”#</code> 中文注释—— <code>#</code> 前少了一个空格，bash 把整行解析成“变量赋值 + 要执行的命令”，于是变量只作用于那条命令的子环境、并未真正赋值。补一个空格即解决。 教训 ： <code>#</code> 前面永远留一个空格。
Ray 装不上，报“依赖无解”	为把 torch 指向 CPU 索引而设的 <code>UV_CONFIG_FILE</code> 被全局导出，导致 Ray 的依赖解析也被限制在 PyTorch 索引里， <code>packaging</code> 找不到合适版本。修法：装 Ray 前 <code>unset</code> 掉该配置。
终端截图总是被裁掉一半	CSS 里 <code>white-space: pre</code> 加在了容器上，于是 HTML 源码中 <code><div></code> 之间的换行符也被渲染成换行，实际行高翻倍。把 <code>pre</code> 只加在每行元素上，并改为“渲染后按内容自动裁剪”，问题消失。

三、对“从 0 到 1 的科研”的一点体会

- 论文里的“我们实现了”，隐含着一整套硬件前提。SGLang 的 CPU 后端在论文与文档里读起来是通用的，但代码事实上把 AVX-512 + AMX 焊死进了编译选项。做复现时，“能不能编译”和“能不能运行”是两个完全不同的问题；前者只证明工具链齐了，后者才验证了前提假设。
- 越是“最不像 bug 的报错”，越要向下层找证据。`Illegal instruction` 没有 Python 栈，看起来很吓人；但反汇编 + `CPUID` 两条命令就把问题钉死在了指令集上。当上层语言给不出信息时，就该下到二进制和硬件那一层去看。

- **能回退的改动才是可靠的改动。** 我的补丁保留了上游分支、备份了原文件、并做了能力探测自动选择路径， 而不是直接把源码改成"我这台机器能跑"的样子。这样同一个脚本在 Xeon 机器上依然走上游最优路径， 结果可被别人复现。
- **"数据里的公共前缀"值得先看一眼再动手。** 我在压测前先统计了 Mooncake trace 的字段，发现 23608 条记录里 `hash_ids[0]` 只有 4 个不同取值、最常见者占 46.3% —— 这说明真实负载天然存在共享前缀。于是压测中让所有请求共享 256 token 前缀， 服务端日志随之出现大量 `#cached-token > 0` (38 个 prefill 批次中 76.3% 命中) 。 **先观察数据，再设计实验，往往比先写代码更省时间。**

四、最终结果

在 WSL2 中以纯 **CPU 推理** (Intel Core Ultra 5 338H, 12 vCPU; 本机核显 Intel Arc B370 未在 WSL 中直通, 故未参与计算) , 成功从源码构建 SGLang 0.5.14 并启动 OpenAI-compatible 服务, 使用 Qwen3-0.6B, RadixCache 正常启用。

以 Mooncake FAST'25 trace 采样 20 条、泊松到达 ($\lambda = 0.5$ req/s) 压测: **成功率 20/20, 总墙钟 72.8 s, TTFT 均值 2113.5 ms, 输出吞吐 12.28 tok/s。**

—— 全部过程、日志与脚本均已随交付物一并保存, 可完整复现。